

Lecture 7 - Database Normalisation

A functional dependency X determines Y means that any two rows agreeing on X must agree on Y . Normalisation is the process of decomposing relations so that the only functional dependencies remaining are those implied by keys.

First normal form requires that every attribute holds a single atomic value. A column storing a comma-separated list of phone numbers violates 1NF, because the individual values cannot be queried or constrained independently.

Second normal form additionally requires that no non-key attribute depends on only part of a composite key. Such a partial dependency causes update anomalies: the same fact is stored in many rows, so changing it requires changing all of them, and missing one leaves the database inconsistent.

Lecture 7 - Third Normal Form and Beyond

Third normal form removes transitive dependencies, where a non-key attribute depends on another non-key attribute. Storing both a postcode and the city it implies is the classic example: the city depends on the postcode, not on the table's key.

Boyce-Codd normal form tightens 3NF by requiring that every determinant be a candidate key. BCNF is strictly stronger than 3NF, and unlike 3NF a decomposition into BCNF is not always dependency-preserving, which is why 3NF is sometimes deliberately kept.

Denormalisation reverses these decompositions on purpose, trading redundancy for read performance. It is a legitimate choice for analytical workloads, where joins dominate cost and the data is written once, but it moves the burden of consistency from the schema to the application.